

Preguntas de Reflexión – Junior Santiago Daza Rodriguez

1. ¿Cuánto tiempo tardaste en entender el código original antes de poder modificarlo? ¿Qué parte fue más difícil de descifrar?

Ya llevo prácticamente 2 años estudiando códigos como este en la carrera, pero no voy a mentir, esté me tomó fácilmente de 4 a 5 horas poder entender mínimamente cómo funcionaba en su etapa inicial.

No entendía que se relacionaba con que, donde iniciaba un método y donde otro, no comprendía que hacían variables como a, b o c, al estar todo junto y comprimido se me cansaba la vista constantemente al tratar de leer el código. Perdía rápidamente el hilo producto de que el código tenía errores de indentado y fue muy tedioso iniciar.

Personalmente hablando, se me hizo complejo entender la clase “**Proceso**” por sus variables que no me permitían comprender que guardaban, ya que no tenían un nombre claro.

```
public static double procesar(double a,double b,double c,double d,double e,int f,boolean g){
double res=0;double iva=0;double prop=0;double tmp=0;
```

Caso que me paso también en la clase **utilidades**.

```
public class Utilidades {
public static double calcular(double pr,double cn,double dc,double iv,double pp,int ni,boolean ap){
double res=0;double tmp=0;double aux2=0;
// calcula el resultado
res=pr*cn;
if(dc>0){res=res-(res*dc);}
tmp=res*iv;res=res+tmp;
if(ap){res=res+(res*pp);}
// imprime restaurante
```

Diría que estas me tomaron mas tiempo y esfuerzo para poder refactorizarlas sin romper el código.

2. ¿Cuál de las 34 malas prácticas te pareció la más peligrosa en un sistema real? ¿Por qué?

Desde mi punto de vista, **el 17**, por razones de seguridad podría ser uno de los más peligrosos, ya que uno lo leería y pensaría que solo valida, pero por dentro está modificando el total, y eso puede pasar por desapercibido durante mucho tiempo.

Pero el **caso 2** me parece el más peligroso en un contexto real, ya que viola las

estructuras de proyectos limpias porque no valora el uso del encapsulamiento, entonces esa clase es como una bolsa con variables globales , a la cual, cualquier clase puede acceder, volviéndola volátil, presta a bugs muy difíciles de detectar y hasta en casos reales podría ser la razón de una vulneración de seguridad que comprometería información valiosa.

**3. ¿Qué pasaría si este sistema tuviera 50.000 líneas en lugar de ~200?
¿Cuánto costaría agregar un nuevo impuesto del 2%?**

Considerando lo que me costó a mí con el código de este tamaño, uno de 50000 líneas se podría tardar varios días, porque se tendría que primero encontrar donde están los valores que manejan los descuentos o impuestos, en montones de clases que usan las mismas variables de forma errónea, luego habría que editar o añadir el nuevo impuesto, verificar en cientos de líneas de código duplicadas que esto no rompa la lógica, aunque la use mal, para luego verificar que todas estas líneas duplicadas tengan el nuevo cambio.

Rápidamente el código se volvería insostenible, por ejemplificar.

Si un cálculo esta repetido y encontrarlo en el código de 200 líneas tomo un tiempo considerable, en uno de 50000 sería algo inviable porque podría estar en cualquier lado.

Incluso se podría tardar semanas solo en entender si el flujo del nuevo impuesto funciona correctamente.

Algo que tomaría unas horas en un código ordenado y limpio se puede convertir en días o incluso semanas de trabajo y búsqueda, haciendo pruebas a mano porque tampoco se cuenta con un espacio de test.

4. Identifica al menos un punto donde podrías haber introducido un bug accidentalmente al refactorizar. ¿Cómo lo detectarías?

De no haber refactorizado variable por variable dependiendo de que función cumplieran, las variables, “a, b, c, d, e, f ” y demás, pueden provocar problemas al establecimiento. Porque sí, el programa funcionaría, compilaría, porque yo puedo

definir que “a” será el precio unitario y “d” será la tasa de IVA, pero si b (que es cantidad) yo la intercambiara por c (que es descuento) el programa simplemente cobraría mal.

Porque sería algo como:

Precio x descuento = \$40.000 pesos * 0.05 = \$2000 pesos (a x c)

En lugar de Precio x cantidad = \$40.000 pesos * 2 = \$80.000 pesos (a x b)

Y como todas estas variables son **doubles**, se harían los cálculos sin problemas, porque son de mismo tipo, si fuesen distintos el compilador avisaría.

Detectar este bug sería sencillo porque se sobre entienden los precios, si en un pedido se pidieron 2 carnes grandes y esta cuesta \$32.000 pesos la unidad, **el subtotal debe dar \$64.000** si da \$1600 el bug es evidente inmediatamente.

5. El método validar() tiene un efecto secundario oculto: además de validar, modifica Datos.tot y Datos.tmp. ¿Por qué este tipo de bugs son especialmente difíciles de encontrar en producción?

Porque el nombre actúa como un camuflaje, al leer **hayProductosEnPedido()** uno lo clasifica como una pregunta, uno no sospecharía la causa de que ahí esta la razón de fallos en el total, uno esperaría que eso pase en **calcularTotal()**.

Otra cosa muy importante es que el bug no ocurre siempre, solo se activa cuando el contador es igual a 0, en producción esto sería un caso específico donde existe una mesa vacía.

6. ¿Cómo conectas este ejercicio con los casos reales de Knight Capital (\$440M en 45 minutos), Southwest Airlines (17.000 vuelos cancelados) o Equifax (147M de registros filtrados) vistos en clase?

Con el caso de Knight Capital, un comportamiento global fue modificado desde un lugar inesperado durante un despliegue. El sistema ejecutó órdenes de compra reales en modo equivocado durante 45 minutos antes de que alguien lo detectara.

Cuando **Datos.tot** es público y estático, cualquier método en cualquier archivo puede cambiarlo. En 200 líneas se nota. En un sistema de trading con millones de transacciones por segundo, para cuando detectas el problema ya perdiste \$440 millones de dólares.

Southwest canceló 17.000 vuelos porque su sistema de asignación de tripulaciones colapsó bajo carga inesperada. La raíz técnica era un sistema heredado donde la lógica de scheduling estaba duplicada y entrelazada en múltiples módulos como las malas prácticas del taller. Cuando intentaron parchear el sistema bajo presión, modificaron una copia del cálculo pero no las otras. El sistema tomaba decisiones contradictorias sobre qué tripulantes estaban disponibles dependiendo de qué módulo consultaba.

imprimirFacturaCompleta() en el código original calculaba el total y actualizaba contadores y cambiaba el estado de la mesa. Tres responsabilidades mezcladas. Bajo presión y con prisa, ese tipo de método es una bomba de tiempo, tocas una parte y rompes otra sin saberlo.

Equifax expuso 147 millones de registros en 2017. Una de las causas técnicas documentadas fue código donde funciones de consulta tenían efectos secundarios que modificaban permisos de acceso y logs de auditoría, cuando una función hace más de lo que su nombre indica, los auditores de seguridad no lo buscan ahí. Los testers no lo prueban. Los desarrolladores no lo sospechan. El bug se esconde en el lugar donde nadie mira.